

Authentication Hardening Blueprint

A reusable task series for building secure auth from scratch (or hardening an existing one)

This is a distilled record of a real authentication security series (T1–T7, with T8 scoped but not yet built) run end-to-end on a production-track SaaS app: Node/Express + MongoDB backend, two Next.js apps (customer-facing + admin), deployed on Cloud Run/Vercel. It captures the **target architecture**, the **task split**, and the **security findings each task closes**, so a future project can hand this whole document to Claude to turn into project-specific task specs — either all at once, or one task at a time in order.

Stack assumption: Node/Express + MongoDB/Mongoose backend, JWT access tokens, one or more Next.js (or similar SPA) frontends. If your stack differs, the *architecture decisions* and *task order* still apply — only the specific APIs/libraries named below (`jose` , `scrypt` , Mongoose `findOneAndUpdate` , etc.) need substituting for your stack's equivalents.

How to use this:

1. Give this file to Claude at the start of a new project (or when hardening an existing one) with: *"Review this against our actual auth code and draft task specs from `templates/TASK_TEMPLATE.md` , following `CLAUDE.md` 's plan → approve → implement → review workflow."*
2. Claude should treat this as a **starting design**, not gospel — it must read the real codebase, confirm what already exists, and adjust the task split/scope accordingly (see the "Verify, don't assume" note in each task below — this bit us more than once in the original run: a task's written scope was 2 days old by the time review happened, because earlier tasks had already shipped fixes the review didn't know about yet).
3. Tasks can be run **in the order given** (each depends on the ones before it — see the dependency line on each task), or a later task's design section can be read standalone if you only need one flow (e.g. just OAuth hardening on an app that already has decent sessions).

0. The five flows this covers

1. **Register → verify email → set password → session** (no password exists until the email is proven).
2. **OAuth sign-in** (Google / GitHub), including linking to an existing account.
3. **Change password** (authenticated, from Settings).
4. **Forgot / reset password** (unauthenticated recovery).
5. **Change email address** (authenticated, requires proving control of the new mailbox).

Plus the cross-cutting concerns that make all five actually secure: session/token architecture, cookies + CSRF, rate limiting, password hashing, and — later — admin role-based access control.

1. Target architecture (build this once, all tasks build on it)

Sessions and tokens

- **Short-lived access JWT** (10–15 min), signed with a per-purpose key (derive with HKDF from one root secret, or use separate secrets outright — never reuse one secret for access tokens, OAuth state, and anything else). Claims: `sub` , `aud` (`customer` | `admin` — **never** a shared audience across a customer-facing and staff-facing app), `iss` , `jti` , short `exp` . Use a real JWT library (`jose` for Node) — do not hand-roll HMAC signing/verification.
- **Opaque rotating refresh token** (32 random bytes, stored hashed) in a `sessions` collection: `userId` , `aud` , `familyId` , `hash` , `createdAt` , `lastUsedAt` , `expiresAt` , `ip / userAgent` , `revokedAt` ,

`revokedReason` . Rotate the refresh token on every use; if a *already-rotated* token is presented again, revoke the whole family (this is the reuse-detection signal for a stolen refresh token).

- **Per-user tokenVersion** bumped on password change/reset, email change, deactivation, or role change. Every request re-checks the access token's embedded version against the current DB value — this is what makes revocation *immediate* even though the access token itself is still technically unexpired. Fail **closed**: if the DB is unreachable, refuse the request, don't trust the token's claims alone.
- Customer sessions can persist ("remember me" — 30 day idle refresh); admin sessions should not persist across browser restarts and should have a much shorter absolute lifetime + idle timeout.

Cookies and CSRF (only relevant if your apps and API are on different origins — common with

Vercel + Cloud Run/Render/Fly)

- Prefer a **same-origin BFF**: your frontend's own server proxies `/api/*` to the real backend, so the browser only ever talks to one origin and cookies are first-party. This lets you use `__Host-` prefixed cookies (`Secure` , `HttpOnly` , `Path=/` , no `Domain` attribute — the strongest cookie hardening available).
- If you can't do a BFF yet (e.g. mobile clients also need this API), support **both** cookie and bearer-token transport behind a single config flag (`bearer` | `cookie` | `both`), so the cutover to cookie-only is a config change later, not a rewrite. Bearer tokens in `localStorage` are XSS-readable — treat `both` / `bearer` as a known, temporary weaker mode, not the end state.
- CSRF defense in layers, all required together: (a) `Origin` / `Sec-Fetch-Site` allowlist, (b) a CSRF token issued alongside the session and required as a custom header on every unsafe request (double-submit or synchronizer pattern), (c) require `Content-Type: application/json` on state changes (blocks classic `<form>`-based CSRF, which can't set that header). `SameSite` alone is not enough — treat it as defense-in-depth, not the whole defense.
- If a request can present **both** a bearer token and a cookie, define an explicit precedence rule (e.g. "bearer wins") and apply it consistently — an inconsistent rule here is itself a vulnerability class (request smuggling between the two identities).

Password hashing and policy

- `bcrypt` or `argon2id` with a **versioned, parameterized hash format** (`$crypt$v=1$n=...,r=...,p=...$salt$hash`) so you can raise cost parameters later and transparently rehash-on-next-login instead of forcing a mass reset. Node's built-in `bcrypt` is fine; just don't use its bare defaults — raise `N` toward OWASP's current guidance and document the chosen values.
- Cap password length (e.g. 128 chars) before hashing — uncapped input is a hashing-cost DoS lever.
- **NIST 800-63B-style policy**, not composition rules: minimum length (10–12), no forced uppercase/digit/symbol rules, screen against a breached-password list (HIBP's k-anonymity range API is the standard free option — you send a hash prefix, never the password or full hash), block the user's own email/name as the password. One shared policy module used by every code path that sets a password (signup, reset, change, admin-set) — duplicated policy logic *will* drift.
- On login with a valid old-format hash, verify it, then transparently rehash into the current format — this is how you raise cost parameters without a forced reset campaign.

Single-use tokens (email verification, password reset, OAuth exchange, email change, invites)

- **One collection, one shape**, not a bespoke field-on-the-user-document per purpose. Fields: `userId` , `purpose` , `audience` , `tokenHash` (never store the plain token), `expiresAt` , `usedAt` , `meta` (purpose-specific payload, e.g. `{currentEmail, newEmail}` for an email-change token — bind everything relevant into `meta` so the consume step can re-validate nothing has drifted since issuance), `sentAt` (for a resend cooldown).
- **Atomic consume**: `findOneAndUpdate({ tokenHash, usedAt: null, expiresAt: { $gt: now } }, { $set: { usedAt: now } })` . A separate "check if valid" then "mark used" is a race condition — two

concurrent requests can both succeed. If a flow needs to *peek* before committing (e.g. showing "confirm this change?" before the user clicks a button), do the peek as a read-only lookup that never sets `usedAt`, and only the actual action does the atomic consume.

- Cap live unused tokens per `(userId, purpose)` (e.g. 3) and enforce a resend cooldown (60s) so a user mashing "resend" doesn't spam mail or fill the collection.
- Different TTLs per purpose: exchange codes ~60s, password reset ~30min, email verification ~24h, a "set your first password" grant ~15min.

Rate limiting

- A **shared store** (Redis, or a TTL-indexed Mongo collection) — an in-memory limiter resets on every cold start and is per-instance, so its real-world limit is `configured_limit × instance_count`, not what you configured.
- Per-**account** limits in addition to per-IP (a distributed attacker with many IPs still only has one target account). Dedicated limits on every credential-touching endpoint — login, password reset, change-password, email-change, OAuth exchange — not just the ones that were obviously abuse-prone at launch.
- Uniform, uninformative responses on auth failures: don't let response time, error text, or status code differ between "wrong password" and "no such account" (constant-time-ish comparison; a dummy hash to compare against for unknown accounts so `bcrypt/script` is *always* run in the login path, not conditionally).

Enumeration resistance

- Signup/registration, forgot-password, and any address-change endpoint should return the **exact same response** (status, body, timing) regardless of whether the target address exists, belongs to someone else, or is closed — do all target-dependent work (issue a token, send a mail vs. send an "account already exists" notice) in the background *after* the response is already sent (`setImmediate` or a queue), never before.

2. Task series

Each task below states its **objective**, the **finding(s) it closes**, its **dependencies**, and the **design decisions** worth carrying forward. Line numbers and file names are deliberately omitted — Claude should locate the real equivalents in your codebase and write them into the actual task spec.

***Verify, don't assume, before writing each task's "Current Behavior" section.** In the original run, later tasks' specs sometimes described the codebase as it existed 1–2 days earlier, because an even-later task's line numbers had shifted or a bug had already been fixed by an unrelated task. Read the actual current code immediately before writing each task, not from memory of an earlier review.*

T1 — Secrets and bootstrap (backend, do first, small)

Objective: Production fails to start with a default/missing/weak signing secret. Separate keys per purpose (access tokens, OAuth state, any other signed-but-not-a-session blob). Lock down whatever creates your very first admin/superadmin account. **Closes:** hardcoded/default JWT secret with no production fail-fast; unauthenticated or racy first-admin bootstrap. **Depends on:** nothing — ship independently. **Carry forward:** a real bootstrap needs *some* one-time secret or out-of-band trigger (env var, CLI seed script) — never "first HTTP request wins," which is racy and, if the bootstrap endpoint is also unauthenticated, exploitable by anyone who reaches your API first.

T2 — Single-use tokens and rate limits (backend)

Objective: Build the single-use token collection and shared rate-limit store described in §1, ahead of using them everywhere. Move to uniform, per-account-aware limits and enumeration-safe responses on login/reset/OAuth-exchange. **Closes:** non-atomic token consume (race conditions); per-instance-only rate limiting; missing per-account lockout; timing/response-shape oracles that reveal whether an account exists. **Depends on:** T1 (shares the secrets

work if your rate-limit keys are HMAC'd). **Carry forward:** decide your test-database strategy *now* — you'll want integration tests that hit a real database for this collection's atomicity guarantees, not a mock. A throwaway, automatically-named test database on your real dev cluster (never the shared dev database) is a good default; an in-memory Mongo works too if your team prefers it, but exercise real atomic `findOneAndUpdate` behavior, not a mocked one.

T3 — Password module (backend + every app with a password form)

Objective: Versioned password hash format with rehash-on-login, NIST-style policy, breach screening, one shared policy module. **Closes:** unparameterized/uncapped password hashing; composition-rule policy with no breach screening; policy logic duplicated (and drifting) across frontend pages. **Depends on:** can run in parallel with T2 if nothing else is racing to change the same user-model fields.

T4 — Sessions, cookies, CSRF (backend + every app) — usually the biggest task, plan to split

Objective: Build the session/refresh/ `tokenVersion` architecture and the cookie+CSRF model from §1. Real logout (actually revokes something) and "sign out everywhere." Security headers (CSP, `Referrer-Policy`, frame protections). **Closes:** no revocation (stolen tokens live until natural expiry); tokens with no audience separation (a customer-app token working against admin endpoints); browser-storage-only tokens with no first-party-cookie option; missing security headers. **Depends on:** T2 (if you reuse its rate-limit/token infrastructure for refresh-token rotation bookkeeping). **Carry forward:** this task is large enough to split into "sessions + audience + revocation" (backend-only) and "cookies/CSRF/transport" (backend + every app), and the latter can itself split per-app if two frontends need the client-side change independently. Plan the split explicitly before starting — don't discover mid-implementation that it should have been two tasks.

T5 — Registration reorder (backend + customer app)

Objective: No password exists until the email address is proven. Register (email only) → verify → single-use "set password" grant → set password → session. Login rejects any account with no password hash. **Closes:** pre-verification account takeover (attacker registers a victim's email with their own password; victim later signs in via OAuth, which proves the email and links the account — inheriting the attacker's password unless you explicitly wipe it, which is the OAuth-linking rule below). **Depends on:** T2 (the single-use-token collection), T3 (the password module for the "set password" step). **Carry forward, product decisions worth deciding up front rather than discovering mid-build:**

- If someone registers an email that already has an account, do you send an "already registered" notice, or silently no-op? (Recommend: notice, no link, so the real owner learns about the attempt, but the response to the *caller* is identical either way — see enumeration resistance.)
- **Forgot-password should also be the recovery path for an account that never finished registration** (verified-but-passwordless, or never even opened the verify link) — don't build a second "resend registration" flow when "forgot password" already reaches the same account and can verify + let them set a password in one step.
- **An OAuth-only account (no password) should be able to add a first password later from Settings**, as an *additional* sign-in method, without an email round trip — the live session already proves the same trust level any other authenticated settings change relies on.

T6 — OAuth hardening (backend + customer app)

Objective: PKCE (S256) if your provider supports it (Google does; many "OAuth App" style integrations like GitHub's classic flow don't — check per-provider), a browser-bound state nonce (`__Host-` cookie) so the authorization response can be tied back to the browser that started it, single-use/atomically-consumed exchange codes, and an explicit rule for what happens when an OAuth sign-in resolves to an email that already has a password. **Closes:** login-CSRF (an attacker can pre-authenticate their own OAuth flow, then trick a victim into completing it, signing the victim into the *attacker's* account); exchange codes that can be replayed or aren't bound to the browser that requested

them; OAuth silently linking into and inheriting an unverified account's stale password. **Depends on:** T2 (token store for the exchange code), T4 (audience separation — a customer-app OAuth sign-in must never be able to resolve to a staff/admin account). **Carry forward — the linking rule, stated explicitly because it's easy to get backwards:**

- OAuth resolving to an email with **no password at all** (a pending/never-finished registration): link normally — the provider just proved the address, there's no credential to inherit.
- OAuth resolving to an email that **has a password but is unverified**: this is the takeover shape from T5 — wipe the password hash, bump `tokenVersion`, revoke sessions, invalidate outstanding tokens, *then* link and mark verified.
- OAuth resolving to a **staff/admin account** from your customer-facing OAuth flow: refuse it entirely (customer OAuth should never be able to touch a staff-audience account) — this needs checking both when initiating the link and again at the exchange step, since state can change in between.
- GitHub in particular: only accept the account's **primary, provider-verified** email — GitHub will hand back unverified or secondary addresses if you're not explicit about which one you asked for.

T7 — Change-password, forgot-password finalization, and email-change (backend + customer app)

Objective: Every credential-touching action (password change, password reset, email address change) is a *complete* security event: re-authenticate the actor, notify the account owner by mail every time, and destroy every session and outstanding single-use token the change should invalidate — not just the one the current action used. **Closes:** password reset/change leaving other sessions and stale tokens alive; silent credential changes (no notification, so a takeover is invisible until the victim is locked out); email address change requiring no re-authentication and no proof of the new address, making a stolen access token sufficient for full account takeover. **Depends on:** T4 (session revocation primitives), T2/T3 (token store, password module). **Carry forward — the email-change design, since it's the most novel piece and easy to get subtly wrong:**

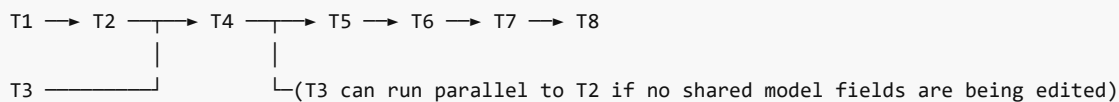
- **Confirm at the new address, before the address moves** — not "switch immediately, verify later." The address only changes when a single-use token *delivered to the target mailbox* is consumed, and that consume is **atomically conditioned on the account's email still matching what it was at request time** (`findOneAndUpdate({ _id, email: originalEmail }, { $set: { email: newEmail }, ... })`) — otherwise a stale token issued before some other change could hijack a since-moved address.
- **Re-authenticate with the current password** (or, for a password-less OAuth-only account, accept the live session as sufficient proof, consistent with T5's "add a first password" decision) before even issuing the token — don't gate only the confirm step.
- **The confirm action needs an explicit user gesture (a button press), not an auto-consuming page load.** A mail scanner or link-preview bot issues a GET; if your confirm page auto-fires the state-changing request on mount, the bot silently burns the token or (worse) completes an irreversible, session-revoking change before the real user ever sees the page. Split it: an anonymous **peek** endpoint (masked target address, no side effects) for the page to render, and a **separate POST**, wired only to a button's click handler, that actually consumes the token.
- **Uniform acknowledgement** on the request step regardless of whether the target address is free, already registered to someone else, or belongs to a closed/staff account — same enumeration argument as registration/forgot-password. Do all target-dependent branching in the background, after the response is already sent.
- On successful confirm: bump `tokenVersion`, revoke every session, delete every other outstanding credential-related token (a stale password-reset or "set first password" grant must not survive an email change), and — this bit us in review — **don't let a client-side "resend" convenience end up depending on a plaintext password or full email address surviving in memory past the submit that used it.** If a form clears sensitive fields after submit (which it must — see below), a "resend" action can't silently reuse them; make it re-prompt.

- Every one of these three flows should send a notification email — to the account's current address on password change, to *both* old and new addresses at the right moments on an email change (old address gets a heads-up when the change is *requested*, so a victim can still act before it completes, and a second notice when it's *done*).
- **Client-side hygiene, easy to miss in review:** clear password fields from component state/memory on every outcome — success, failure, *and* a thrown network error — not just the happy path. A form that only clears on success leaves a plaintext password sitting in memory (inspectable via browser devtools) after every failed attempt.

T8 — Admin role-based access control (backend + admin app) — scoped here, not yet built in this reference

Objective: Move from a single coarse "admin" role to explicit permission strings (`users.manage` , `billing.read` , ...), resolved server-side per request, never trusted from a JWT claim. Reserve the most dangerous actions (create/edit other admins, change permissions, deactivate/reactivate staff) for a super-admin tier. Add an invite flow for new staff (no admin-chosen plaintext password — a single-use, audience-scoped invite link, same token-store mechanism as everything else). Append-only audit log for admin actions. Consider MFA (TOTP) as a requirement for staff accounts specifically, given the blast radius of a compromised admin account. **Closes:** coarse role grants every admin every capability; reactivating a closed account has no role check; an admin-created staff account starts with an admin-chosen password instead of proving mailbox ownership; no audit trail for who did what to which account. **Depends on:** T4 (audience separation), T5 (the password-hash/no-password patterns an invite flow reuses), T7 (the notification-email patterns an invite/reset-by-admin flow reuses). **Note:** this task was scoped but intentionally not built in the reference run this document is based on — treat its objective/closes/depends-on as a starting design, expect it to need more back-and-forth on the permission list and invite-flow specifics than the other tasks did.

3. Task ordering and parallelism



- T1 can ship alone, immediately.
 - T2 and T3 can run in parallel **if** you assign clear ownership of any shared model file they'd both touch (typically your `User` model) — otherwise one task's migration/field addition can silently undo the other's mid-flight edit.
 - Everything from T4 onward is sequential in practice: each depends on session/audience primitives the previous task built, and T5/T6/T7/T8 all touch overlapping surface (the user model, the token store, the mail templates) closely enough that running them in parallel invites merge conflicts more than it saves time.
 - If your team genuinely needs T5 and T8 in parallel (both touch the user schema), use `templates/PARALLEL_EXECUTION_TEMPLATE.md` and give exactly one of them ownership of any schema migration.
-

4. What "done" looks like for the whole series

- No password can come into existence for an unproven email address.
- Every session-issuing or session-revoking action goes through one shared set of primitives (`tokenVersion` bump + session revocation), never a bespoke one-off.
- Every single-use token (verify, reset, set-password, OAuth exchange, email-change, invite) lives in one collection with atomic consume — no bespoke "is this token still good" logic anywhere else.

- No credential-changing endpoint leaves stale sessions or stale tokens behind, and none of them are silent — the account owner always gets a mail.
 - No response anywhere in the auth surface reveals whether a target email address exists, except to someone who has already proven they control it.
 - Every rate limit is account-aware and backed by a store that survives a cold start / spans every instance.
 - A customer-facing token can never reach an admin-facing endpoint, and vice versa.
-

5. Notes from running this the first time (things that cost real back-and-forth)

- **A finding closed by an earlier task will make a later task's spec describe stale behavior** if the later task's author works from the original review instead of the live codebase. Always re-verify "Current Behavior" against the actual code immediately before writing a task spec, and say so explicitly in the spec ("read from `<branch>` on `<date>` , not from the N-day-old review").
- **Two review findings can point at the same code from opposite directions** and look contradictory until you realize the real fix is a small redesign, not a patch to either one in isolation. (Concretely: "a form field isn't cleared on failure" and "a convenience feature relies on that same field surviving between submits" are the same bug wearing two hats — the fix is to stop the convenience feature from depending on retained state, not to pick a side.)
- **Live-verify UI fixes in an actual browser, not just by reading the diff** — a re-entrancy guard fix that stops a duplicate network request can still leave the UI itself hung if an unrelated cleanup function (e.g. a stale-closure cancellation flag) discards the one request's result. Code review alone did not catch this in the original run; a live click-through did.
- **Treat an implementer's "I tested X" as a claim, not a fact**, especially for anything security-relevant — re-run the actual commands yourself, and for anything requiring a live server, drive it yourself. This cost nothing when the claim was accurate and caught real gaps when it wasn't (an untested transport mode, a claimed-fixed lint warning that was actually pre-existing and unrelated, etc.).